Autoescalamento Horizontal de Pods em Kubernetes: uma Análise Comparativa entre Implantações Local (Minikube) e em Nuvem (AWS EKS)

Ivo Motoki Makiyama Lopes[1]

[1]Escola Politécnica – Universidade de São Paulo (USP)
Disciplina PSI5120 – Tópicos de Computação em Nuvem

ivo.motoki@usp.br

Código-fonte, manifestos, scripts e evidências completas disponíveis em:
https://github.com/IvoMotoki/psi5120-trabalho-hpa-final

Abstract. *This paper presents the design, deployment, and evaluation of a web server running under Kubernetes Horizontal Pod Autoscaler (HPA), deployed in two environments: a local Minikube cluster and a managed AWS EKS cluster. Using an identical CPU-bound workload and load-generation method in both environments, we measured HPA reaction time, maximum replica count reached, scale-down time, approximate cost, and operational complexity. The Minikube deployment scaled from 1 to 10 replicas (the configured maximum) in about 2 minutes and 10 seconds, unconstrained by node capacity. The EKS deployment, running on Free-Tier-eligible* `t3.micro` *nodes, reacted even faster (roughly 51 seconds to the first extra replica) but was capped at 5 replicas by a per-node pod limit unrelated to CPU or memory, illustrating a structural difference between a resource-bound local cluster and a managed, quota-bound cloud environment.*

Resumo. *Este artigo apresenta o projeto, a implantação e a avaliação de um servidor web sob autoescalamento horizontal de pods (HPA) no Kubernetes, implantado em dois ambientes: um cluster local via Minikube e um cluster gerenciado na AWS EKS. Usando a mesma carga de trabalho, intensiva em CPU, e o mesmo método de geração de carga nos dois ambientes, medimos o tempo de reação do HPA, o número máximo de réplicas atingido, o tempo de retração, o custo aproximado e a facilidade de gerenciamento. A implantação no Minikube escalou de 1 para 10 réplicas (o máximo configurado) em cerca de 2 minutos e 10 segundos, sem restrição de capacidade dos nós. A implantação no EKS, rodando em nós* `t3.micro` *elegíveis para o nível gratuito da AWS, reagiu ainda mais rápido (cerca de 51 segundos até a primeira réplica extra), mas ficou limitada a 5 réplicas por um limite de pods por nó independente de CPU ou memória, evidenciando uma diferença estrutural entre um cluster local limitado por recursos de hardware e um ambiente gerenciado na nuvem limitado por cotas de infraestrutura.*

1. Introdução

O autoescalamento horizontal é um dos mecanismos centrais que tornam a computação em nuvem atrativa para cargas de trabalho com demanda variável: em vez de provisionar

capacidade fixa para o pico de uso, o sistema ajusta automaticamente o número de instâncias de um serviço em resposta a métricas observadas, como utilização de CPU. No ecossistema Kubernetes, esse mecanismo é implementado pelo *Horizontal Pod Autoscaler* (HPA), um controlador que monitora periodicamente métricas de recursos dos pods de um *Deployment* e ajusta o número de réplicas dentro de limites mínimo e máximo configurados, de forma a manter a utilização média próxima de um valor-alvo.

Embora o HPA seja conceitualmente simples, seu comportamento prático depende fortemente do ambiente em que o cluster Kubernetes está implantado. Um cluster local, como os providos pelo Minikube, tem sua capacidade determinada apenas pelos recursos de CPU e memória alocados à máquina virtual ou contêiner que hospeda o cluster. Já um cluster gerenciado em nuvem, como o Amazon Elastic Kubernetes Service (EKS), introduz camadas adicionais de infraestrutura – rede virtual, balanceadores de carga, integração com o provedor de identidade e acesso, e principalmente o tipo de instância EC2 usado nos nós – que podem impor restrições de capacidade independentes da CPU ou memória efetivamente disponíveis.

Este trabalho tem como objetivo projetar, implantar e testar o mesmo servidor web, sob a mesma política de HPA – seguindo o roteiro oficial de autoescalamento do Kubernetes [Kubernetes Authors 2026] e a documentação de uso do HPA no Amazon EKS [Amazon Web Services 2026] –, em duas implantações distintas: (A) um cluster Kubernetes local via Minikube e (B) um cluster gerenciado na AWS EKS. A carga de trabalho e o método de geração de estresse foram mantidos idênticos nas duas implantações, de modo a isolar as diferenças de comportamento que decorrem do próprio ambiente de execução. A análise compara o tempo de reação do HPA, o número máximo de réplicas efetivamente atingido, o tempo de retração após o fim da carga, o custo aproximado de cada implantação e a facilidade de gerenciamento observada durante o processo.

O restante deste artigo está organizado da seguinte forma: a Seção 2 revisa os conceitos de autoescalamento horizontal no Kubernetes; a Seção 3 descreve a arquitetura e a topologia das duas implantações; a Seção 4 detalha a metodologia de implantação e de geração de carga; a Seção 5 apresenta os resultados obtidos e a análise comparativa; e a Seção 6 traz as conclusões e considerações finais.

2. Autoescalamento Horizontal de Pods no Kubernetes

O HPA é um controlador nativo do Kubernetes que atua sobre um *Deployment* (ou outro objeto com escala, como um *ReplicaSet* ou *StatefulSet*), ajustando o campo `replicas` periodicamente com base em métricas coletadas de cada pod. No modo mais comum, a métrica usada é a utilização de CPU, expressa como percentual do valor pedido em `resources.requests.cpu` de cada contêiner: se a média de utilização entre os pods prontos ultrapassa o percentual-alvo configurado, o controlador calcula um número maior de réplicas necessário para trazer a utilização média de volta ao alvo, respeitando o limite `maxReplicas`; se a utilização cai abaixo do alvo, o número de réplicas é reduzido, respeitando o limite `minReplicas`.

Duas características do HPA são especialmente relevantes para os resultados deste trabalho. Primeiro, o controlador depende de uma fonte externa de métricas de recursos – tipicamente o *metrics-server*, um componente que coleta periodicamente dados de uso de CPU e memória de cada nó/pod via `kubelet` e os expõe pela API de métri-

cas do Kubernetes (`metrics.k8s.io`). Sem o metrics-server instalado e funcional, o HPA não consegue calcular a utilização e permanece com o valor `<unknown>` no lugar do percentual-alvo. Segundo, o HPA aplica políticas assimétricas de estabilização: o aumento de réplicas (*scale-up*) reage rapidamente (por padrão, a cada ciclo de sincronização do controlador, tipicamente da ordem de 15 segundos), enquanto a redução de réplicas (*scale-down*) respeita, por padrão, uma janela de estabilização de 5 minutos contada a partir da última recomendação de réplicas elevada – uma medida para evitar oscilações (*flapping*) quando a carga é intermitente. Essa assimetria é observada de forma consistente em ambas as implantações deste trabalho.

É importante distinguir o autoescalamento horizontal do autoescalamento vertical (*Vertical Pod Autoscaler*, VPA): enquanto o HPA aumenta ou diminui o *número* de réplicas de um pod, mantendo os requests/limits de cada réplica fixos, o VPA ajusta os próprios requests/limits de CPU e memória de um pod já em execução. Este trabalho usa exclusivamente o HPA, por ser o mecanismo mais diretamente comparável entre uma implantação local e uma implantação em nuvem gerenciada.

3. Arquitetura e Topologia das Soluções

3.1. Implantação A – Minikube (local)

A Implantação A utiliza o Minikube (versão 1.38.1) para criar um cluster Kubernetes de nó único, executado sobre o driver Docker, dentro de uma distribuição WSL2 (Ubuntu 24.04) em uma máquina Windows 11. O perfil do cluster (`hpa-2026`) foi criado com 3 vCPUs e 4096 MB de memória alocados à máquina virtual do Minikube. O *metrics-server* foi habilitado via o addon nativo do Minikube (`minikube addons enable metrics-server`), que já aplica internamente os ajustes necessários para funcionar em um ambiente de desenvolvimento local (como a validação relaxada de certificados TLS do kubelet). Toda a carga de trabalho foi implantada no *namespace* dedicado `tf-hpa`. Por se tratar de um cluster local de nó único, a capacidade disponível para o autoescalamento é determinada exclusivamente pela CPU e memória alocadas à máquina virtual – não há um segundo nível de quota de infraestrutura, como ocorre em um provedor de nuvem.

3.2. Implantação B – AWS EKS (nuvem)

A Implantação B utiliza o Amazon EKS, um serviço gerenciado de Kubernetes, provisionado via `eksctl` (versão 0.230.0) na região `us-east-1`. O cluster (`psi5120-tf-hpa-eks`, Kubernetes 1.34) é composto por um *control plane* totalmente gerenciado pela AWS e um *node group* gerenciado com instâncias EC2 do tipo `t3.micro`. A escolha do tipo de instância não foi arbitrária: a conta AWS utilizada está sujeita a uma restrição de elegibilidade para o nível gratuito (*Free Tier*), rejeitando a criação de instâncias `t3.medium` originalmente planejadas com o erro `InvalidParameterCombination: not eligible for Free Tier`. O node group foi dimensionado com 4 nós – um ajuste feito após observar que, com apenas 2 nós, a capacidade de pods por nó do `t3.micro` (ver Seção 5.2) era insuficiente até mesmo para agendar os próprios pods de sistema do EKS, sem sobrar espaço para o servidor web sob teste.

O EKS instala automaticamente, como *addons* gerenciados, os componentes `vpc-cni` (rede de pods via ENIs da VPC), `kube- proxy`, `coredns` e, relevante

para este trabalho, `metrics-server` – diferentemente do Minikube, em que o metrics-server precisa ser explicitamente habilitado como addon. O acesso externo à aplicação foi provido por um `Service` do tipo `LoadBalancer`, que a AWS provisiona automaticamente como um Classic Load Balancer (CLB) com um nome DNS público, permitindo confirmar o funcionamento da implantação a partir de fora do cluster.

4. Metodologia

4.1. Aplicação Alvo e Manifestos

Em ambas as implantações, o alvo do autoescalamento é o mesmo: um *Deployment* chamado `php-apache`, usando a imagem oficial de exemplo do *walkthrough* de HPA do Kubernetes (`registry.k8s.io/hpa-example`) – um servidor PHP/Apache com um endpoint que executa um cálculo intensivo de CPU a cada requisição, permitindo gerar carga de CPU de forma previsível e proporcional ao volume de requisições. O contêiner é configurado com `requests.cpu = 200m` e `limits.cpu = 500m` (`requests.memory = 64Mi`, `limits.memory = 128Mi`); o valor de `requests.cpu` é a base sobre a qual o HPA calcula o percentual de utilização. O objeto `HorizontalPodAutoscaler` associado usa a versão `autoscaling/v2` da API, com `minReplicas: 1`, `maxReplicas: 10` e uma métrica de recurso do tipo `Utilization` com alvo de 50% sobre `requests.cpu`. Usar exatamente o mesmo par de manifestos (Deployment e HPA) nas duas implantações é o que torna a comparação entre elas metodologicamente válida: qualquer diferença de comportamento observada decorre do ambiente de execução, não da carga de trabalho em si.

A exposição do serviço difere ligeiramente entre as implantações por necessidade prática: no Minikube, o `Service` é do tipo `ClusterIP` (não há balanceador de carga de nuvem disponível localmente); no EKS, um segundo `Service`, também do tipo `LoadBalancer`, foi adicionado especificamente para demonstrar o acesso público – sem alterar o Deployment ou o HPA usados no teste de carga em si.

4.2. Geração de Carga

A carga de estresse foi gerada por um conjunto de pods `busybox` executando, em laço infinito, requisições HTTP (`wget`) contra o `Service` `php-apache` pelo nome DNS interno do cluster (`php-apache.tf-hpa.svc.cluster.local`). Gerar a carga de dentro do próprio cluster, resolvendo o serviço pelo DNS interno, garante que o mesmo mecanismo de teste seja usado nas duas implantações, independentemente da topologia de rede externa de cada uma. O script de automação (`scripts/load_test.sh`) sobe os pods geradores, registra um retrato do estado do HPA e dos pods a cada 10 segundos por um período de carga ativa (240 segundos), remove os geradores de carga e continua registrando por um período adicional (360 segundos) para observar a retração. O número de pods geradores usado foi de 6 no Minikube e de 3 no EKS – a redução no EKS decorre diretamente da limitação de capacidade de pods por nó discutida na Seção 5.2, e não de uma escolha metodológica arbitrária.

5. Resultados e Análise Comparativa

5.1. Implantação A – Minikube

A Tabela 1 resume os principais momentos do ciclo de autoescalamento observado no Minikube.

Tabela 1. Ciclo do HPA no Minikube

Momento	Hora	Réplicas	CPU (alvo 50%)
Antes da carga	02:38:27	1	8%
Início do scale-up (CPU)	02:39:38	1	192%
1º scale-up	02:39:49	4	192%
Scale-up intermediário	02:41:47	8	167%
Pico (máximo configurado)	02:41:59	10	167%
Carga removida	02:42:35	10	131%
CPU baixa, sem retração ainda	02:44:28	10	10%
Retração completa	02:54:40	1	26%

O tempo entre a carga atingir os pods geradores (aproximadamente 02:38:39) e o primeiro aumento de réplicas (02:39:49) foi de cerca de 1 minuto e 22 segundos. O HPA atingiu o número máximo de réplicas configurado (`maxReplicas: 10`) em aproximadamente 2 minutos e 10 segundos desde o início da carga, sem qualquer indício de limitação de capacidade – a única variável que limitou o escalonamento foi o próprio parâmetro `maxReplicas`, não a CPU ou memória disponíveis na máquina virtual do Minikube. Após a remoção da carga (02:42:35), a utilização de CPU caiu rapidamente (10% já às 02:44:28), mas o número de réplicas só retornou a 1 às 02:54:40 – um intervalo de aproximadamente 12 minutos entre o fim da carga e a retração completa, consistente com a janela padrão de estabilização de *scale-down* do HPA (5 minutos), contada a partir da última leitura de utilização elevada, e não a partir do instante em que a carga foi removida.

5.2. Implantação B – AWS EKS

A Tabela 2 resume o ciclo equivalente observado no EKS.

Tabela 2. Ciclo do HPA no AWS EKS

Momento	Hora	Réplicas	CPU (alvo 50%)
Antes da carga	01:06:57	1	8%
Início do scale-up (CPU)	01:07:31	1	116%
1º scale-up	01:07:48	3	248%
Pico (limitado por capacidade)	01:08:05	5	185%
Carga removida	∼01:10:57	5	–
CPU baixa, sem retração ainda	01:16:59	5	8%
Retração completa	01:18:05	1	10%

No EKS, o tempo de reação foi ainda menor que no Minikube: cerca de 34 segundos entre a carga atingir os pods e o primeiro salto de utilização (116%), e cerca de 51 segundos até a primeira réplica extra ser criada (01:07:48). Esse tempo de reação mais rápido é compatível com o fato de o `metrics-server` do EKS já estar em execução como addon gerenciado desde a criação do cluster, sem a janela inicial de "aquecimento"de coleta de métricas observada no Minikube logo após a implantação.

O aspecto mais relevante da Implantação B, porém, não é a velocidade de reação, e sim o *teto* de escalonamento efetivamente atingido. Mesmo com a utilização de CPU medida em 248%/50% e 185%/50% – valores muito acima do necessário para justificar réplicas adicionais dentro do limite configurado de `maxReplicas: 10` – o HPA não conseguiu escalar além de 5 réplicas. A causa não é de CPU ou memória, e sim um limite estrutural do tipo de instância: nós `t3.micro` suportam no máximo 4 pods por nó, um limite imposto pelo número de endereços IP disponíveis por *Elastic Network Interface* (ENI) na implementação da VPC CNI da AWS, e não pelos recursos computacionais do nó. Com os *daemonsets* do próprio sistema do EKS (`aws-node` e `kube-proxy`, um pod de cada por nó) já ocupando parte dessa capacidade, e com os pods `coredns` e `metrics-server` (2 réplicas cada, por padrão) disputando o restante, a capacidade líquida disponível para o `php-apache` e para os próprios pods geradores de carga, mesmo com o node group ampliado para 4 nós, ficou restrita a poucas unidades – valor consistente com o teto de 5 réplicas observado.

Esse comportamento contrasta diretamente com o Minikube, onde a única restrição de capacidade é a CPU/memória alocada à máquina virtual do cluster, sem um segundo nível de cota de infraestrutura. Trata-se de uma diferença estrutural entre um cluster local e um cluster gerenciado em nuvem operando sob restrição de custo: o mesmo HPA, com a mesma política e sob uma carga proporcionalmente maior, produz resultados de escalonamento qualitativamente diferentes dependendo exclusivamente do tipo de instância disponível.

Quanto à retração, a janela de observação do script de teste (360 segundos após a remoção da carga) quase capturou o evento de *scale-down*, mas terminou minutos antes; a retração completa (5 para 1 réplica) foi confirmada manualmente às 01:18:05, também consistente com a janela padrão de estabilização de 5 minutos do HPA – o mesmo comportamento assimétrico observado no Minikube.

5.3. Custo Aproximado

A Implantação A tem custo direto nulo, por rodar inteiramente em recursos de hardware já disponíveis na máquina do desenvolvedor. A Implantação B incorre em custo real de infraestrutura AWS: a taxa fixa do control plane gerenciado do EKS (aproximadamente US$0,10/hora, sem cobertura do nível gratuito) e o custo das instâncias EC2 do node group (`t3.micro`, tipicamente elegíveis ao nível gratuito da AWS dentro do limite de 750 horas/mês combinadas, ou aproximadamente US$0,0104/hora cada, fora desse limite). O cluster da Implantação B permaneceu ativo por aproximadamente 63 minutos, do provisionamento à exclusão completa confirmada, resultando em um custo aproximado de US$0,10 a US$0,15 para todo o experimento – a maior parte proveniente do control plane, que não tem nível gratuito, independentemente do tempo de uso.

5.4. Facilidade de Gerenciamento

A implantação local via Minikube foi operacionalmente mais simples de iniciar – um único comando cria o cluster e o addon de metrics-server em poucos minutos –, mas exigiu atenção a recursos do host: uma tentativa inicial de criação do cluster falhou de forma intermitente devido a esgotamento de memória RAM da máquina Windows (menos de 500 MB livres), causando reinícios sucessivos do *control plane* local até a memória ser

liberada. Esse tipo de instabilidade de ambiente é uma classe de problema que praticamente não existe em um cluster gerenciado, onde o control plane roda em infraestrutura da AWS, isolada da máquina do desenvolvedor.

A implantação no EKS, por outro lado, exigiu etapas adicionais de configuração ausentes no fluxo local: instalação de ferramentas de linha de comando específicas (`aws-cli`, `eksctl`), configuração de credenciais de acesso programático via IAM (distintas das credenciais de login no console web), e um tempo de provisionamento consideravelmente maior – cerca de 15 a 20 minutos apenas para o *control plane*, mais alguns minutos para o *node group*, contra menos de um minuto no Minikube. A conta utilizada também impôs uma restrição não documentada de antemão (elegibilidade a instâncias do nível gratuito), que só se manifestou como um erro durante a criação do node group e exigiu ajuste do tipo de instância e, por consequência, do dimensionamento do cluster. Por fim, o encerramento correto de uma implantação em nuvem exige disciplina adicional – excluir o `Service` do tipo `LoadBalancer` antes do cluster, para evitar que o balanceador fique órfão, e verificar explicitamente, via linha de comando, que nenhum recurso cobrável permaneceu ativo –, uma preocupação operacional inexistente no Minikube, onde basta apagar o perfil local.

6. Conclusão

Este trabalho implantou e testou o mesmo servidor web sob a mesma política de Horizontal Pod Autoscaler em duas implantações Kubernetes distintas – Minikube local e AWS EKS gerenciado – usando exatamente a mesma carga de trabalho e o mesmo método de geração de estresse, de forma a isolar as diferenças de comportamento decorrentes do próprio ambiente de execução.

Os resultados mostram que o mecanismo de autoescalamento reage de forma qualitativamente semelhante nos dois ambientes – aumento rápido de réplicas (da ordem de dezenas de segundos a poucos minutos) seguido de uma retração deliberadamente mais lenta (da ordem de vários minutos, ditada pela janela de estabilização padrão do HPA) –, mas que o *teto* efetivo de escalonamento pode ser determinado por fatores completamente alheios à política do HPA em si. No Minikube, esse teto foi o próprio `maxReplicas` configurado, atingido sem qualquer restrição adicional. No EKS, o teto real foi imposto por um limite de pods por nó associado ao tipo de instância elegível ao nível gratuito da AWS – uma restrição de infraestrutura que não tem equivalente direto em um cluster local, e que só se torna visível na prática, e não na documentação do HPA.

Do ponto de vista de facilidade de gerenciamento, o Minikube favorece a iteração rápida e o baixo atrito inicial, ao custo de depender inteiramente da estabilidade e da capacidade de hardware da máquina local. O EKS, por sua vez, oferece um control plane gerenciado e isolado, mas introduz complexidade operacional adicional – configuração de credenciais, tempo de provisionamento maior, restrições de conta e cuidados redobrados de limpeza de recursos para controle de custo – que devem ser considerados ao planejar cargas de trabalho de produção sensíveis a custo e a capacidade de escalonamento.

Como trabalhos futuros, seria valioso repetir o experimento em instâncias EC2 de maior capacidade de rede (fora do nível gratuito), de forma a isolar o efeito do limite de pods por nó observado neste trabalho, e avaliar também o *Cluster Autoscaler* do Kubernetes em conjunto com o HPA, permitindo que o número de nós do cluster gerenciado

se ajuste automaticamente à demanda de pods, e não apenas o número de réplicas do Deployment.

Referências

Amazon Web Services (2026). Scale pod deployments with horizontal pod autoscaler. `https://docs.aws.amazon.com/eks/latest/userguide/horizontal-pod-autoscaler.html`. Acesso em: 2026.

Kubernetes Authors (2026). Horizontal pod autoscaler walkthrough. `https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/`. Acesso em: 2026.

A. Apêndice: Trechos das Evidências Brutas

Esta seção reproduz trechos representativos dos arquivos de log brutos gerados automaticamente durante os experimentos (comando/saída de `kubectl get hpa` e `kubectl get pods`, capturados a cada 10 segundos pelo script `scripts/load_test.sh`), correspondentes aos momentos resumidos nas Tabelas 1 e 2. Os arquivos completos (com todas as capturas intermediárias e o estado de cada pod) estão disponíveis no repositório do projeto, nos diretórios `evidencias/minikube/` e `evidencias/eks/`.

A.1. Implantação A – Minikube

Antes da carga (02:38:27):

```
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
php-apache Deployment/php-apache cpu: 8%/50% 1 10 1 4m59s
```

Primeiro scale-up (02:39:49):

```
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
php-apache Deployment/php-apache cpu: 192%/50% 1 10 4 6m22s
```

Pico – máximo configurado atingido (02:41:59):

```
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
php-apache Deployment/php-apache cpu: 167%/50% 1 10 10 8m31s
```

Carga removida (02:42:35):

```
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
php-apache Deployment/php-apache cpu: 131%/50% 1 10 10 9m8s
```

Retração completa (02:54:40, confirmada manualmente após o fim da janela de observação do script):

```
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
php-apache Deployment/php-apache cpu: 26%/50% 1 10 1 21m
```

A.2. Implantação B – AWS EKS

Antes da carga (01:06:58):

```
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
php-apache Deployment/php-apache cpu: 8%/50% 1 10 1 20m
```

Primeiro scale-up (01:07:48):

```
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
php-apache Deployment/php-apache cpu: 248%/50% 1 10 3 20m
```

Pico – limitado por capacidade de pods por nó, não por `maxReplicas` (01:08:04), com réplicas do `php-apache` presas em `Pending` por falta de capacidade agendável:

```
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
php-apache Deployment/php-apache cpu: 185%/50% 1 10 5 21m
NAME READY STATUS RESTARTS AGE NODE
load-generator-7f8684cc8f-5q6sp 1/1 Running 0 69s ip-192-168-39-22.ec2.internal
load-generator-7f8684cc8f-k7h7g 1/1 Running 0 69s ip-192-168-28-221.ec2.internal
load-generator-7f8684cc8f-qxbbv 1/1 Running 0 69s ip-192-168-28-221.ec2.internal
php-apache-56cf58f979-9sbx4 0/1 Pending 0 37s <none>
php-apache-56cf58f979-lbwfj 0/1 Pending 0 22s <none>
php-apache-56cf58f979-lzn77 1/1 Running 0 8m23s ip-192-168-39-22.ec2.internal
```

Retração completa (01:18:05, confirmada manualmente após o fim da janela de observação do script):

```
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
php-apache Deployment/php-apache cpu: 10%/50% 1 10 1 31m
```

A.3. Evidência de Acesso Externo (EKS)

Confirmação de que a aplicação implantada no EKS estava acessível publicamente pela internet, via o DNS do Load Balancer provisionado pelo `Service` do tipo `LoadBalancer`:

```
$ curl -i http://a06f3a000408841fcbfd80005be8ce5f-992750954.us-east-1.elb.amazonaws.com/

HTTP/1.1 200 OK
Date: Wed, 26 Aug 2026 03:55:41 GMT
Server: Apache/2.4.10 (Debian) PHP/5.6.14
X-Powered-By: PHP/5.6.14
Content-Length: 3
Content-Type: text/html; charset=UTF-8

OK!
```